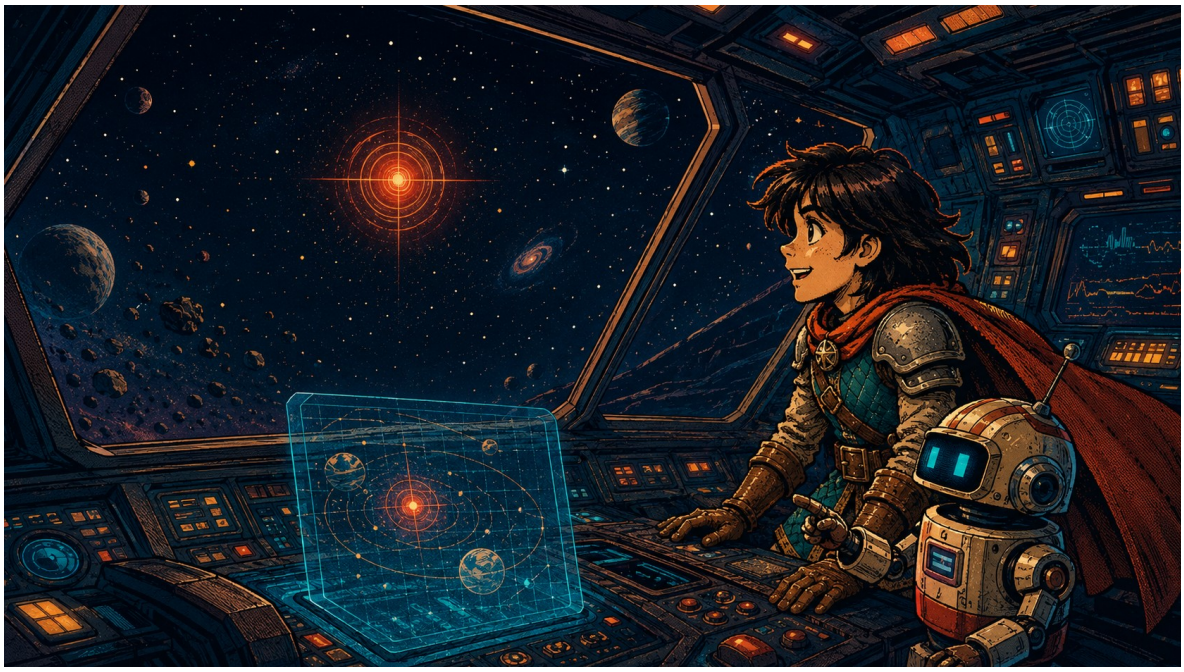
AN INTERACTIVE JOURNEY THROUGH LINKED LISTS

# Mapping Uncertainty

## *One Link at a Time*

*A story of scattered worlds, changing routes, and the addresses that hold them together*



**Lijo M. Jose**

Assistant Professor, School of Computer Science and Engineering

# Before We Begin: You Do Not Have a Map

Hello, fellow explorer.

Before we speak about linked lists, pause for a moment and imagine that Mission Control has handed you a coordinate. Just one. No route. No final destination. No reliable count of how many safe worlds exist beyond it.

Would you draw the entire journey in advance?

You cannot. The neighboring planet may already belong to another civilization. The next one may be wrapped in storms. A world rejected yesterday may become available tomorrow after its occupants leave. Even the bot cannot know where its next landing will be until it reaches the present one and looks again.

So this is not a chapter that begins with a definition and asks a story to decorate it. We will let the expedition develop. Each time the mission asks a new question, we will pause. You will try the tempting answer. We will see where it fails. Only then will we name the structure that the mission has quietly forced us to build.

**A SMALL PROMISE** Do not rush to memorize the arrows. First decide which route must survive. The code will have somewhere meaningful to stand after that.

## How to read this journey

Read with a pencil nearby. Whenever the words ask you to pause, sketch only what is currently known. Do not draw the next planet before the bot discovers it. When a link changes, cross out the old arrow before drawing the new one. That tiny habit exposes most linked-list mistakes long before a compiler can.

You will meet C code because the expedition eventually needs a precise implementation. Our main functions avoid double pointers: whenever an operation can change the first node, it returns the updated head, and the caller stores it. A brief side note near the end shows the double-pointer form so that other programs will still feel readable.

## A quiet prerequisite check

The journey assumes that an address can be stored in a pointer, that -> reaches a field through a structure pointer, that malloc creates storage whose lifetime can outlive a function call, and that free ends that lifetime. If those ideas are familiar—even if not yet comfortable—you are ready.

***Now let us return to that single coordinate. The signal is still waiting.***

PART ONE

# The Coordinate That Was Not a Destination

*The map begins with what nobody knows*

*One coordinate is enough to begin a journey. It is not enough to describe one.*

## 1. The Signal

The signal arrived shortly before dawn.

It contained no map, no list of safe worlds, and no promise that the journey would end after three landings or three hundred. It contained one coordinate and a request: send the Explorer Bot first.

Caitlyn looked at the coordinate again. 'Where does it go after that?'

Mission Control had no answer. That was precisely why the bot was going.



*The Explorer Bot leaves with one known coordinate and an unknown route.*

Imagine that you are designing the bot's memory. You might reserve a neat row of places for the route: first planet here, second planet beside it, third beside that. It feels orderly. It also assumes that the universe will cooperate.

But the planet beside the first may already belong to another civilization. The next may be inaccessible. A third may be free but unsafe. A world rejected during the first survey may become

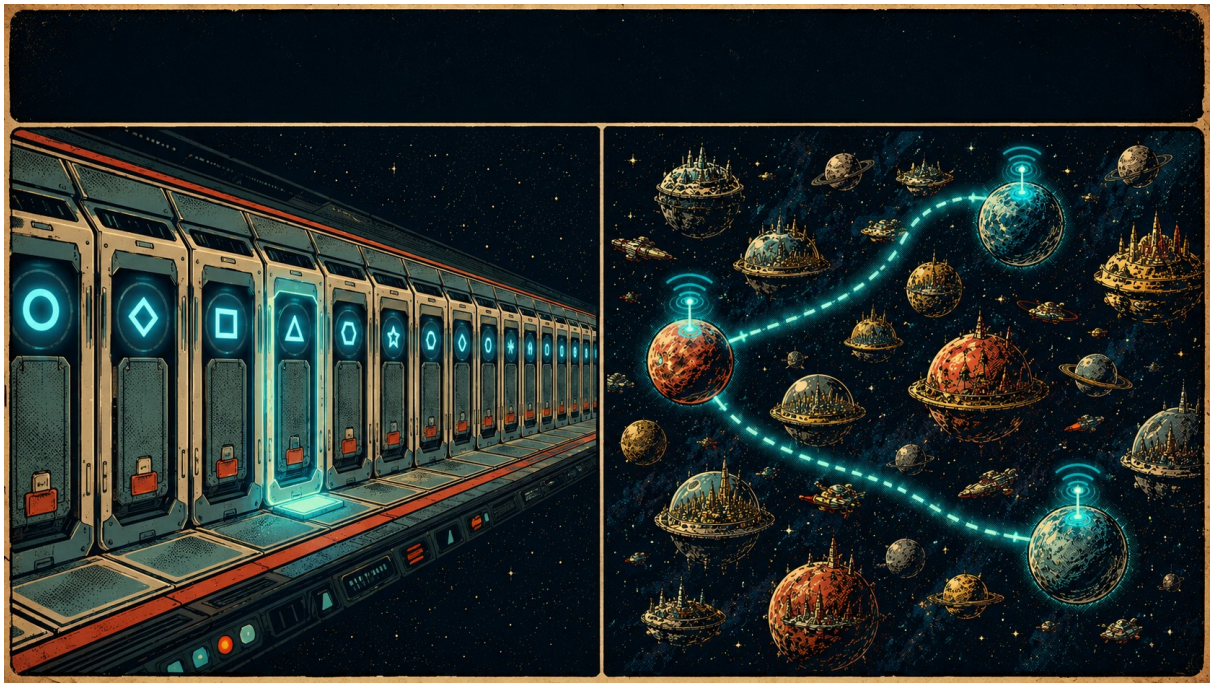
available later after its occupants leave. The order in which worlds become usable has nothing to do with where they sit.

**PAUSE BEFORE CONTINUING** If the second usable planet is physically far from the first, what information would let Caitlyn still treat it as the second stop?

## 2. When Position Stops Meaning Order

In an array, position does much of the storytelling. If the first element begins at a known address, the next is found by moving a fixed number of bytes. The elements are neighbors in memory, so arithmetic can recover element i immediately.

The expedition cannot make that promise. Its usable worlds appear wherever space is available. We therefore need a different way to say 'after this one, go there.' The route must carry its own order.



*The array trusts physical adjacency; the expedition stores the next coordinate explicitly.*

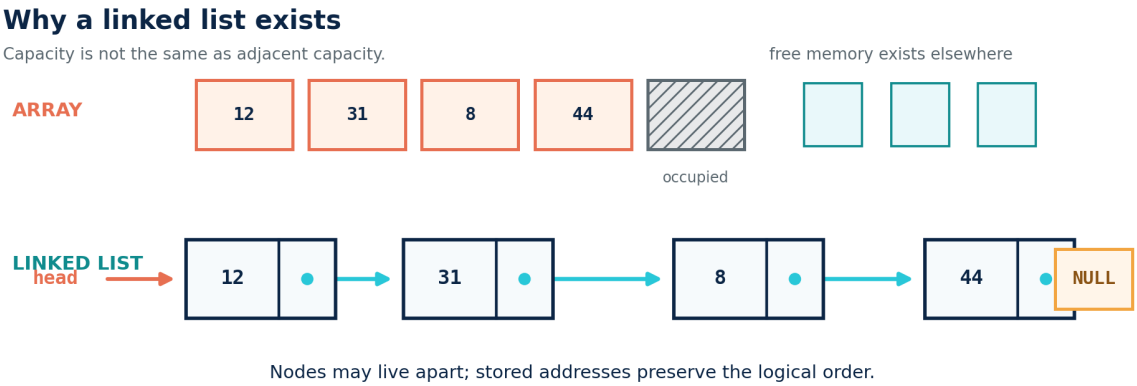



*Figure 1. Adjacency defines array order; stored addresses define linked-list order.*

This does not make arrays inferior. Arrays are excellent when the size is reasonably known, compact storage matters, and direct indexing is frequent. The linked route solves a different problem: uncertain growth among separately available locations.

Notice what changed. We are no longer asking, 'Where is the next position?' We are asking, 'What address has the present stop recorded as next?'

# 3. The Beacon with an Empty Compartment

The bot lands on the first usable world. It collects an observation—perhaps a mineral reading, a danger score, or simply an identifier. Caitlyn will need that information later. She will also need the coordinate of the next usable world.

So the bot installs a beacon with two compartments:

one compartment for the observation;

one compartment for the address of the next beacon.

There is only one problem. The bot has not discovered the next world yet.

Should it leave the second compartment uninitialized and hope to repair it later? No. An uninitialized pointer is not 'unknown' in the safe sense; it contains an indeterminate bit pattern that may look like a real address. The bot needs an honest marker for 'there is no known next stop.' In C, that marker is NULL.



*The first beacon is complete before the route continues: its data is valid and its next field is NULL.*

Only now, after the need is visible, do we give the beacon its C name: a node.

*One node stores one observation and one route to another node*

```
typedef struct Node {
    int data;
    struct Node *next;
} Node;
```

Read the second field slowly. next does not contain another Node. It contains the address of another Node. If the structure contained a complete Node inside itself, that Node would contain another complete Node, and the definition could never end. A pointer has a fixed size, so the structure remains finite.

# 4. The First Coordinate Becomes head

The bot may leave many beacons, but Mission Control needs only one external entry into the route: the address of the first beacon. We store that address in a pointer named head.

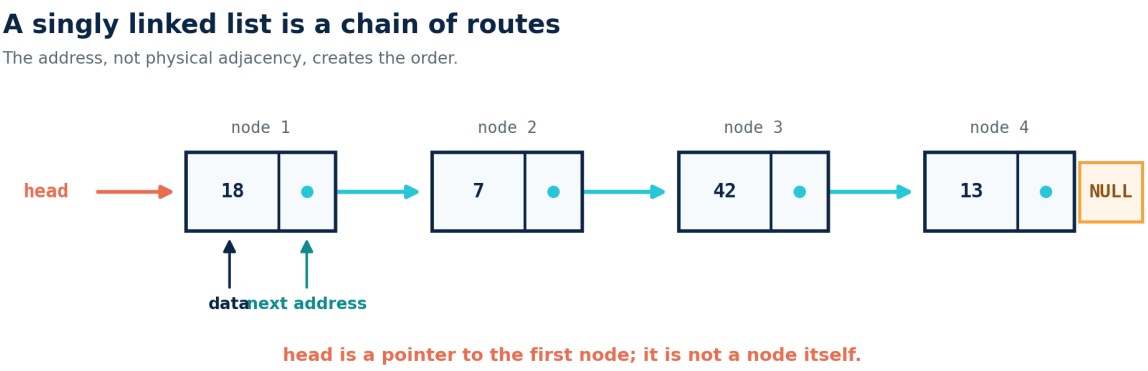


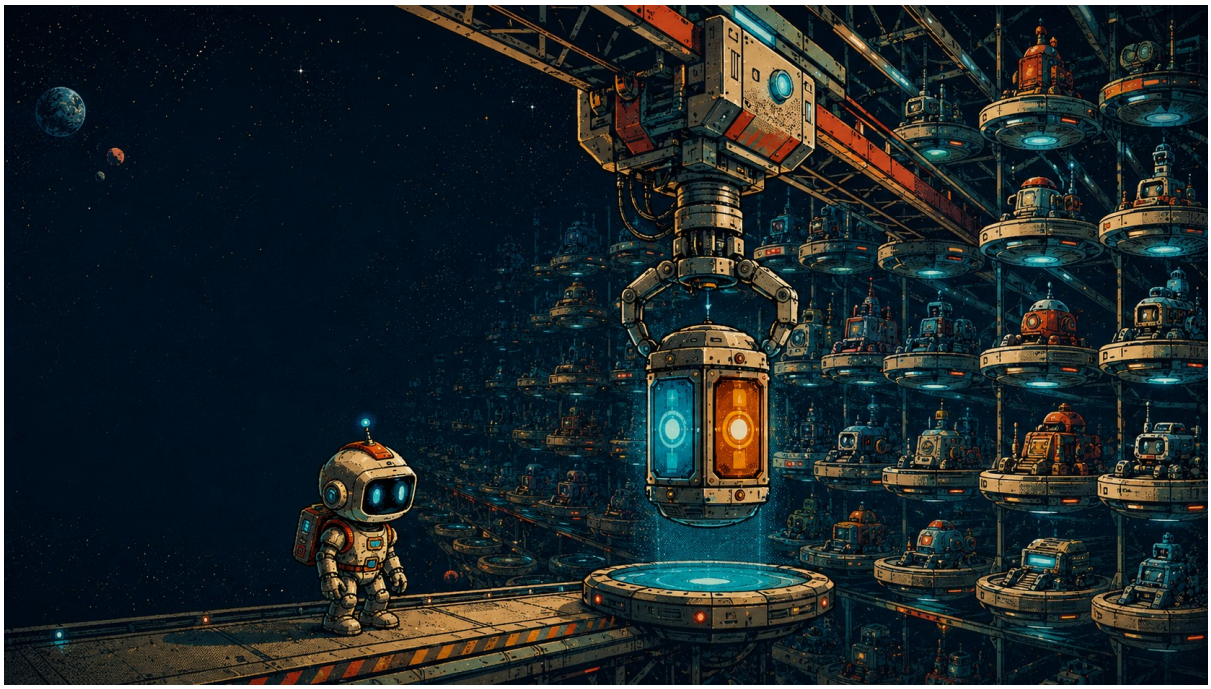

*Figure 2. head is the entry address; each next field continues the route; NULL marks its present end.*

Three names now have three different jobs. Node is a type describing one beacon. head is a Node * variable holding the first beacon's address. head->next is the next field inside that first beacon. Separating those roles makes the route much less mysterious.

And if no beacon exists yet? head is NULL. The empty list is not a special invisible object. It is simply the absence of an entry route.

# 5. Where Does a New Beacon Live?

A local variable inside a function would disappear when that function returns. The beacon must remain after the installation call ends, because Caitlyn may arrive much later. We therefore request storage whose lifetime continues until the program explicitly releases it.



*The allocator provides one suitable node-sized location; it does not promise a neighboring address.*

*Allocate first, verify the address, initialize both fields, then return the new node*

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int data;
    struct Node *next;
} Node;

Node *createNode(int value)
{
    Node *newNode = malloc(sizeof *newNode);

    if (newNode == NULL) {
        fprintf(stderr, "Memory allocation failed.
");
        exit(EXIT_FAILURE);
    }

    newNode->data = value;
    newNode->next = NULL;
    return newNode;
}
```

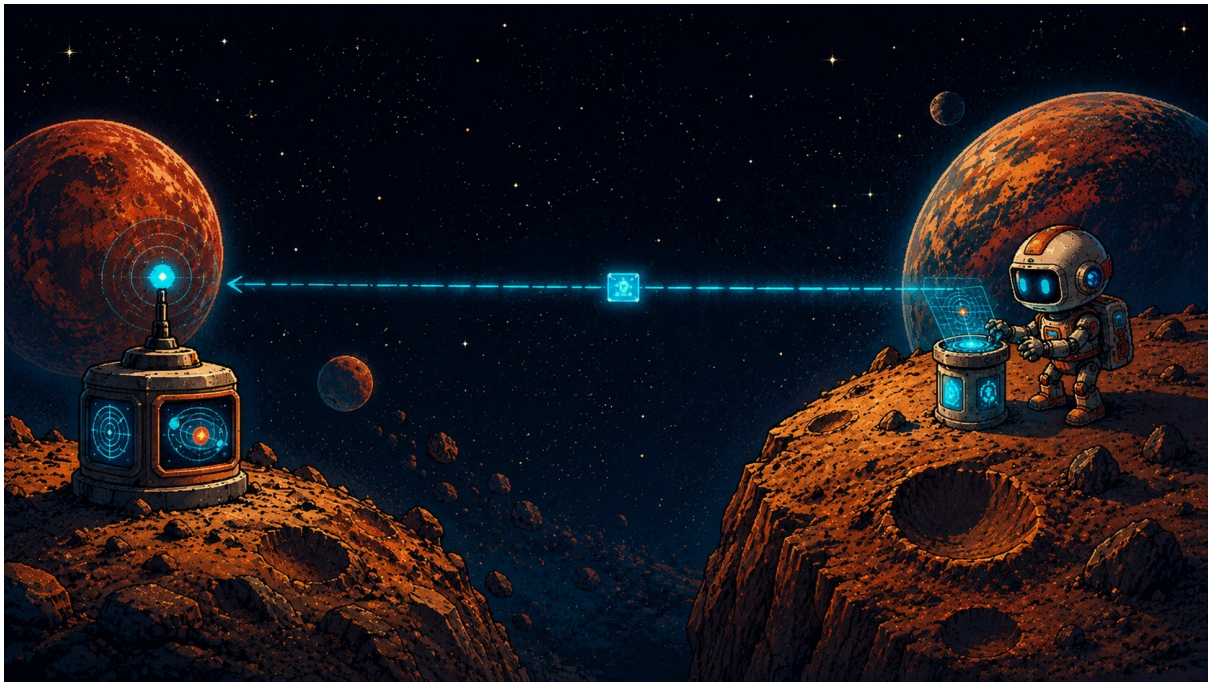
malloc may choose any suitable free region. That freedom is the point: a new node does not have to sit beside the old node. sizeof *newNode follows the declared pointer type, and C converts malloc's void * result to Node * without an explicit cast.

**NOTICE WHAT IS ALREADY SAFE** The function never publishes a half-built node. Allocation is checked, data is stored, and next is set to NULL before the address is returned.

## 6. The Route Grows Behind the Bot

The bot scouts again. A nearby world is occupied, so it passes. Another is inaccessible. At last it finds a usable planet at an address that could not have been predicted earlier. It installs and initializes the new beacon there.

Now look back at the previous beacon. Its next field still says NULL. The new beacon exists, but the journey does not yet reach it. One final action makes the discovery part of the route: the previous beacon stores the new beacon's address.



*Discovery happens first; only then does the previous beacon publish the new coordinate.*

The order matters. First obtain and initialize the new node. Then update the old last node. If allocation fails, the existing route remains honest. If the old link were changed before a valid destination existed, Caitlyn could be sent into nowhere.

At this point the expedition has quietly constructed its first singly linked list: a sequence of independently stored nodes in which each node knows only the next node.

# 7. Caitlyn Follows

The bot has gone ahead. Caitlyn receives head and nothing else. That is enough. She lands at head, reads the observation, follows next, and repeats until the address is NULL.

Would you move head itself as Caitlyn travels? If head advances from the first beacon to the second, what address will Mission Control use to begin again? The route may still exist, but its entry has been forgotten.

So Caitlyn gets a travelling pointer of her own: current. current moves. head stays home.

*Traversal protects the entry route and moves only a temporary pointer*

```
void display(const Node *head)
{
    const Node *current = head;

    while (current != NULL) {
        printf("%d -> ", current->data);
        current = current->next;
    }
    printf("NULL
");
}
```

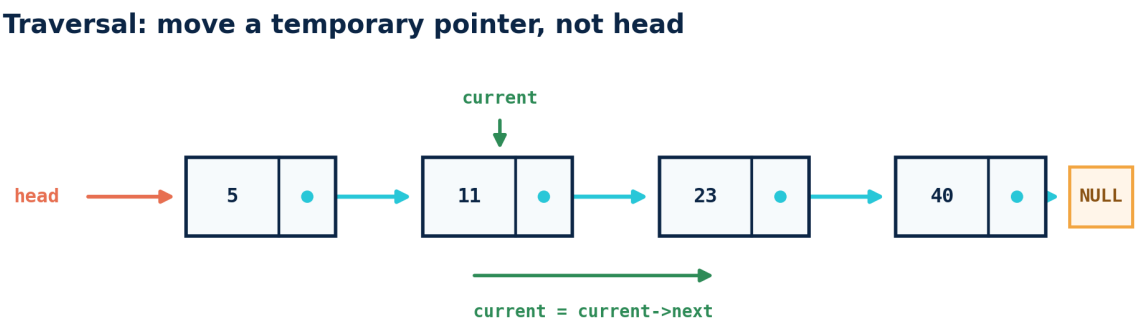



*Figure 3. current advances through the chain while head continues to identify the first node.*

The loop checks current before using ->. The const in const Node * records another promise: this traversal may read the beacons, but it will not rewrite them.

**READING CHECKPOINT** A singly linked list answers one question beautifully: where next? It still cannot answer where did I come from? or how do I return home? Those questions have not been solved because the mission has not required them yet.